

Project Executable Files

Date	10 July 2026
Team ID	xxxxxx
Project Name	Raising Waters
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	NA
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

[Paste or describe your folder structure here. Example:]

```

flood-prediction-app/
├── static/
│   ├── css/
│   │   └── style.css    # Frontend layout & color configurations
│   └── js/
│       └── main.js      # Asynchronous fetch and DOM updates
├── templates/
│   └── dashboard.html   # User interface control dashboard
├── data/
│   └── prediction_logs.csv # Local CSV operational logging file
├── models/
│   └── model.pkl         # Pre-trained classifier weight matrix asset
├── app.py               # Main Flask application routing engine
├── requirements.txt      # Environment library dependency configuration
└── README.md            # Setup and execution instructions documentation
  
```

/project-root

/src - Source code files

/tests - Test files

/docs - Documentation

/config- Configuration files

README.md- Setup and run instructions

requirements.txt - Python dependencies (or package.json for Node)

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://flood-prediction-app-kvfz.onrender.com/
Login Credentials (Demo)	NA (Open public access control interface)
Platform / Hosting Provider	Render Cloud Web Services
Repository Link	https://github.com/reshmashaik-r11/flood-prediction-app
Demo Video Link	https://drive.google.com/file/d/10sXGu1DLcsx320dK5XkcJ05n_UgW0j0L/view?usp=drivesdk

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Step 1: Clone the repository to your local system layout:

```
git clone [https://github.com/reshmashaik-r11/flood-prediction-app.git](https://github.com/reshmashaik-r11/flood-prediction-app.git)
```

Step 2: Open the project terminal directory and install all required framework packages:

```
pip install -r requirements.txt
```

Step 3: Start up the Flask engine web interface node directly:

```
python app.py
```

Step 4: Launch a web browser page and point it to the local system dashboard link:
[\[http://127.0.0.1:5000\]\(http://127.0.0.1:5000\)](http://127.0.0.1:5000)

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Initial Render Cold Start Latency: Because the platform runs on a free cloud hosting tier, the site might take 50–60 seconds to spin up and load if it has been inactive for a while.	Status: Normal platform behavior. A simple page refresh wakes up the web service worker engine immediately.
2	Array Dimensional Input Parsing: Typing strings or irregular symbols into the input fields causes server script mapping errors.	Workaround: Added basic HTML5 value field constraints to only accept proper numeric values for rainfall depth and cloud coverage.
3	Persistent Logging Across Deployments: Running inside a basic Render container environment means entries added to prediction_logs.csv reset whenever the cloud service auto-deploys.	Status: Kept as a localized design choice. For permanent enterprise auditing, the system structure is ready to link up to a dedicated database instance.